Autonomous Driving System Based on Visual Servoing

Abraham De Alba Franco[1], Alejandro Flores Madriz[1], Francisco Ramírez Aldaco[1], and Pablo Agustín Ortega-Kral[1]

Instituto Tecnológico de Monterrey, Guadalajara, México.

Abstract. Autonomous driving in an urban environment is one of the biggest challenges engineering faces, as it requires knowledge from various domains and has numerous safety considerations. In the following paper, we present a basic autonomous driving system that uses visual servoing to detect the center of a lane and follow it using a PID controller as well as a CNN-based image classifier for detecting traffic signs and triggering an appropriate action. Additionally, we use a Kalman Filter to help reduce noise and the detected center.

Keywords: Self-driving, YOLOv5, Kalman Filter, Puzzlebot.

1 Introduction

Self-driving has been a long-standing goal in engineering, dating back to the late seventies, with the first system being proposed by the University of Tsukuba [1]. Essentially, this problem can be divided into 3 main parts: 1) sense the surroundings to identify the road and any relevant context, 2) produce a control action based on the sensed information, and 3) fit the computing power into a manageable package.

Computer vision has been instrumental in the perception of a vehicle's surroundings, mainly because of the high amount of information that can be obtained from a single sensor. The early implementations of self-driving cars used classical computer vision to segment the lanes in the image, as well as classify the markings on the road, whereas modern implementations usually use machine-learning approaches.

Regarding computing power, single-board computers (such as the Raspberry Pi and NVIDIA Jetson) and the reduction of the size of processors/GPUs have made embedding self-driving solutions accessible not only to research labs but also viable for companies and, to a smaller scale, educational prototypes.

The following paper seeks to present a self-driving system specifically designed to solve the challenge put forth by Manchester Robotics, which consists in traversing a 'b' shaped course where the roads have the edge and center lines marked. Additionally, the car will have to be able to identify traffic lights and seven traffic signs based on the UK road signals.

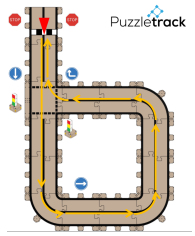

Fig. 1: Manchester Robotics Challenge

In this solution, we propose a Hough Transform-based line detection for center estimation that uses a Kalman Filter as a linear state estimator to mitigate detection noise. For following the center of the lane, we propose a PID controller that acts on the angular velocity of the robot. Regarding sign detection, we implement a YOLOv5-based image classifier.

2 Methodology

2.1 System Overview

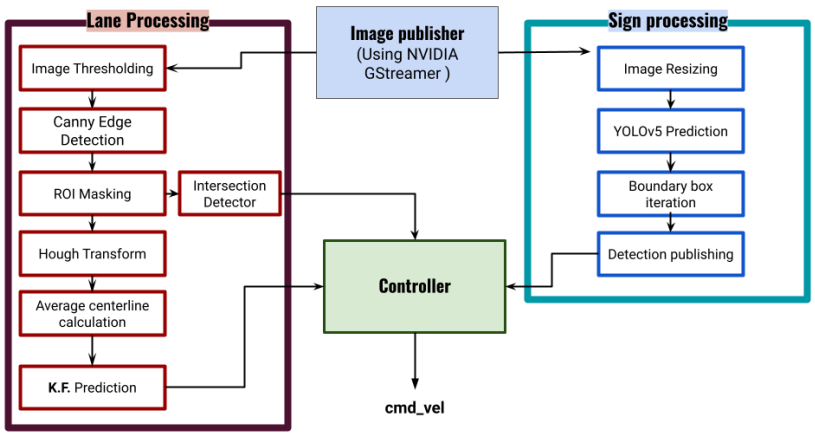

Fig. 2: System block diagram

The system was built for Manchester Robotics' *Puzzlebot*, which is designed for use with ROS. This is a differential mobile robot that has an ESP32 (the "Hackerboard") running proprietary firmware to control the robot's actuators and interface them using the rosserial package. For the vision part, Puzzlebot

has a Raspberry Pi camera read over SCI and an Nvidia Jetson where we will run the control and computer vision algorithms.

As mentioned before, the system is built on ROS. We propose 4 nodes to orchestrate the system, as seen in figure 2. The image publisher node is taken from NVIDIA, it reads the Raspberry Pi camera using GStreamer and publishes as a ROS topic using CvBridge. Following ROS design principles, our nodes are loosely coupled, meaning we have separated the task so that each node only does one thing. In this sense, the lane processing node is in charge of finding the center of the image and publishing the offset of said center based on the robot's POV; the sign processing node runs the image through the YOLOv5 network and returns the detected sign in the image. Finally, the control node applies a constant linear velocity and determines the angular velocity using a PID controller, where the reference is the detected center and the error is the current offset; the output of the controller is a *twist* object.

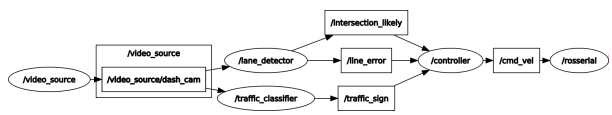

Fig. 3: Node Structure of System

2.2 Lane Processing

A foreword on Kalman Filters The Kalman Filter is a powerful and versatile tool in control theory, and as such it has a variety of uses. In this context, we apply it as a linear state estimator. The diagram for this can be seen in the following image.

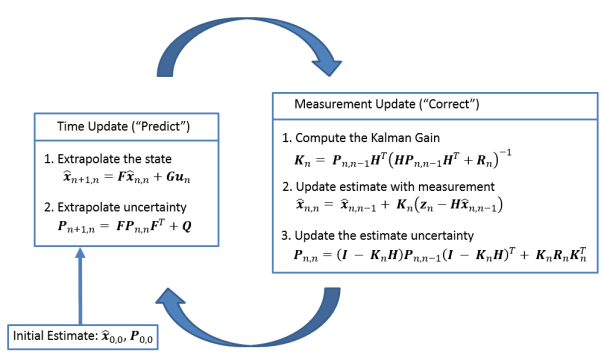

Fig. 4: Diagram of Kalman Filter [2]

As seen in figure 4, the Kalman Filter follows a two-step cyclical process: prediction and correction. For this, it uses the covariance of the process and of the model to determine how much to "*trust*" each measurement and thus propose the best estimate. In each estimate, the filter updates the error of the measurement and the estimate and thus determines a gain. It is worth noting, that for our implementation we used OpenCV's Kalman Filter with wrapper functions to integrate to our lane detector object.

Lane Detection Regarding the actual detection of the lane itself, we achieve this in five steps. First, we apply inverse thresholding over the gray scaled image, setting 75 as a cutoff value, such that any value higher than this will be set as black, and lower values will be white. The intention behind this is to exclude the parts of the image that are not black, knowing that the lane lines are a dark color. Additionally, in this step, we do an opening morphological operation in order to mitigate any reflections and small lines (such as the ones between pieces of track) that may be present.

Having the binary image, we then apply Canny edge detection to get the borders present in it. Canny Edge detection works by using the Sobel operator to find the magnitude and orientation of the gradients within the image, marking the places where the gradient is high as an edge. It also does an additional cleanup step by applying non-maximum suppression and thresholding.

After obtaining the edge, we mask out the region of interest. In the case of the challenge, given that the road has the center line painted, we can mask out the borders of the image as well as the top. This will allow us to only analyze the lines that comprise the center lane marker.

To get the coordinates of the lines in the image, we use the Hough Probabilistic Transform to project the points to the parameter space, where each represents a specific slope and intercept; the probabilistic transform implements an accumulator to divide the parameter space into a grid and reports the parameters corresponding to the biggest number of intersections. For our purposes, we use OpenCV's implementation, which directly reports a vector with all the coordinates of the detected lines.

It is at this point in the lane detection pipeline that we determine if the robot is seeing a street or an intersection. We do this by setting a threshold on the number of lines in the current frame, with the prior knowledge that the markings in the edges of the intersection will result in a higher count, and the absence of lines in the middle of the intersection will cause the count to be 0. If the number of lines exceeds a certain limit, the robot will flag that it is probably at an intersection. The control algorithm does not take this prediction immediately though; it adds or subtracts a "vote" from a counter variable depending on whether an intersection is detected, and runs the intersection action only when this variable reaches a threshold value.

To measure the center, we average the x coordinate of the detected lines and use as the y an arbitrary height. We can do this as we have defined a narrow region of interest, leveraging the center line painted on the track. Defining a narrow region of interest and using a relatively "low" height for a measured center helps us tackle the curved section well, as we essentially approximate the curvature with a lot of straight lines.

While the projected center does work well, we observe in testing that it can "bounce" around due to noise in the image. To avoid this, we implement a Kalman Filter as a linear state estimator to help get the best estimate of the lane center. Finally, we compare the position of the center to the center of the robot's POV to determine the offset from the lane center in pixels, this is the error we publish.

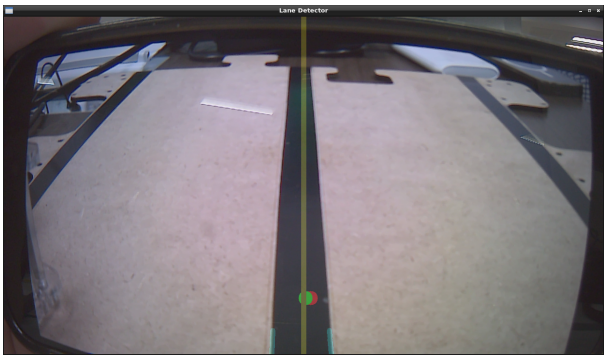

Fig. 5: Lane Detection on the Jetson

2.3 Sign Processing

For the sign/traffic light detector, we trained a YOLOv5 model [3] using a custom database that can be found in appendix A. We chose YOLOv5 as our classifier because the data augmentation process baked into its training gives it good generalizability; additionally, YOLOv5 provides a bounding box around the detected object(s) which allow for further processing based on the perceived size of the signals.

To collect the database, a simple script was written that uses the Puzzlebot's camera to capture and store frames. While doing this, we sought to vary the conditions in which the signals were seen, varying parameters such as lighting and signal positioning in the frame; the intention of this is to contribute to the overall robustness of the model.

The model was trained using PyTorch and then converted to ONNX to use with OpenCV on the NVIDIA Jetson. The detector loads the trained model using

OpenCV's dnn module. Each frame is resized to 640x640 to comply with the YOLO format, and then converted into a blob and passed to the model. For each frame, the model returns a set of predictions, which are then sorted by confidence. To determine which prediction to publish, a minimum size must be met to ensure the signs are not too far into the background. Another implementation detail is that we implement a buffer variable to store the last observed sign, due to the fact that when the robot reaches the intersection, the sign is not visible anymore.

Fig. 6: Classification of signs with YOLOv5

2.4 Controller

The controller has two modes of operation: line follower (closed loop) and open loop position control. The system switches between the two depending on what is happening in the current frame.

The line follower is the default mode of operation. Here, the control is applied over the linear and angular speed of the robot. For the linear part, the robot will maintain a constant speed (determined arbitrarily) affected by a multiplier which will change depending on the direction of the last forward or turn sign detected. This way, the robot can stop or slow down as needed without the rest of the control sequence. We can express said control as $Vel(s) = M(s) * 0.0616$ where s is the detected sign and $M(s)$ is defined as:

$$M(s) = \begin{Bmatrix} 1, & \text{if } s = \text{green}, \\ 0.5, & \text{if } s = \text{yellow}, \\ 0.5, & \text{if } s = \text{construction} \\ 0, & \text{if } s = \text{stop} \\ 0, & \text{if } s = \text{red} \end{Bmatrix} \tag{1}$$

The line follower also calculates angular velocity using a PID controller, where the reference is 0, the error is the offset to the center and the output is the velocity in radians per second. In order to temper the control velocity and limit it within a reasonable range, we implement a sigmoid function such that the max control speed is $\pm 1.12 \frac{rad}{s}$. The control expression for the angular velocity is thus as follows:

$$\omega(e(t), s) = M(s) A_m \tanh\left(k_p e(t) + k_i \int e(t)dt + k_d \frac{de}{dt}\right) \tag{2}$$

It is worth noting that the control constants were determined experimentally and are stored as parameters in ROS, which are loaded at launch time. Another quirk of our implementation is that we used boolean flags to detect if the controller is receiving information from the lane detector and sign classifier, so as to avoid the accumulation of error while the video feed is starting up.

3 Experiments

We tested various iterations of our methodology until finding the best configuration.

3.1 Lane Detector Methods

For the lane detector, we initially implemented a version that masked out the center lines and calculated the center based on the perceived edges. The intention behind this was to develop a more generalizable solution, as not all roads have the center line as clearly marked. Even though we were able to develop a crude baseline, for the purpose of the challenge of Manchester Robotics we decided to use the center detection based on the center line.

Another parameter we tuned was the size and shape of the mask. Originally, we proposed a trapezoid-shaped mask, with the intention of conserving more of the lane in order to have better detection of the curves. However, the extra field of view resulted in more noise and in the end did not yield good results. Because of this, the final mask was narrow and short, as this helps limit the field of view of the robot to a near point and thus discretizes the curves.

Finally, we also tested the impact of the Kalman Filter by running the lane detection without it. We indeed observed that the Kalman Filter helps return the best estimate of the center, which is especially apparent in challenging scenarios when there is more noise in the picture (i.e. under or over-exposure).

3.2 Classifier Optimizations

Regarding sign classification, we tested various iterations of the dataset to ensure each class was appropriately represented, and the model had a wide variety of challenging scenarios. For the first attempt, we only classified traffic lights, with the caveat of adding a class for considering cases when the camera sees a changing traffic light (two lights on at the same time). This approach resulted in poor performance and high training time, as the "changing" classes were severely underrepresented because they constitute a very rare scenario. The resulting confusion matrix (figure 7) shows that the model tended to confuse the classes with each other. For the final implementation (figure 9), we removed the changing classes and ensured an equal representation of the objects being classified.

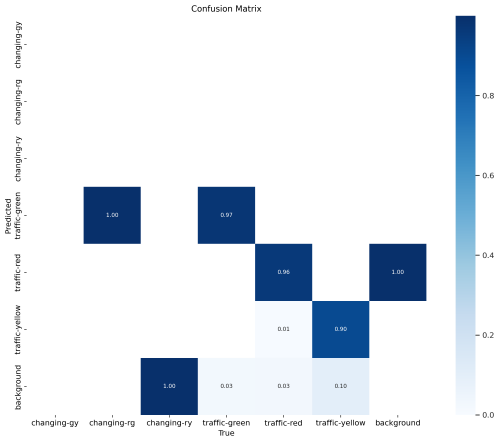

Fig. 7: First Attempt

We also tested the speedup of inference given by transcoding our classifier from Python to C++. Initially, we hypothesized that the improvement would not be significant, as OpenCV is already written in C++ and Python just calls functions from said library. When testing, we were able to receive three times more FPS (∼2 FPS in Python versus ∼6 FPS in C++).

3.3 Implementation Challenges

While implementing the solution, there were some considerations worth noting. Firstly, as ROS 1 is reaching the end of its life cycle, many of the versions of dependencies it uses do not support modern features. This proved challenging, especially when considering the computer vision aspect, as the OpenCV version installed on the Jetson was not compiled with CUDA support enabled, and the deep learning module was not compatible with some operators found in

the YOLOv5 model. Because of this, we recompiled a newer version of OpenCV (4.5.5) with CUDA support enabled. In order to instruct ROS to use this version, we added the vision_opencv package to our project and modified the requirements of the CMakeLists files.

Another big challenge was RAM, as the Jetson model used only has 2 GB, and when it ran out and the device had to use swap memory instead (which is taken from the SD card), the performance took a big hit. To remedy this issue, we removed any visualization of the detected lines/signs and ran the Jetson in minimal mode, while also clearing the cache between runs.

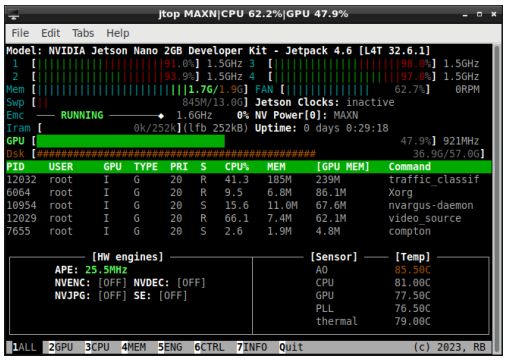

Fig. 8: Resource Usage

4 Results

The proposed system is able to detect the center line and follow it while simultaneously detecting pertinent signals and traffic lights, as can be seen in appendix C. The Kalman Filtering and the sigmoid bounding help the control be smooth and consistent. The size classification of the signals allows for a correct judgment of the relevant indications and prevents the system from taking action too preemptively.

The system shows a fair amount of robustness, largely due to the variety of scenarios provided in the recollected dataset for sign classification. That said, the lane detector does show difficulties in filtering out strong glares.

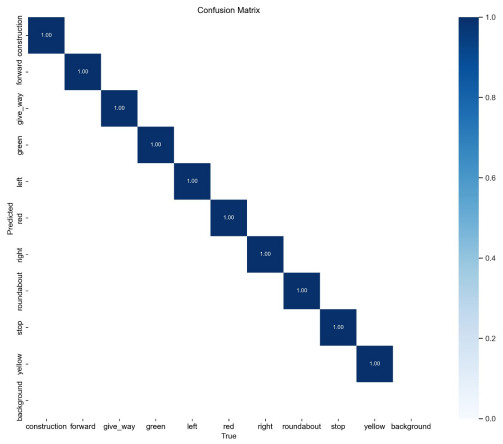

Fig. 9: Final Confusion Matrix

5 Ethical Reflection

Self-driving is a field rife with ethical conundrums. The broader discussion focuses around the necessity of self-driving cars and poses the question: is this something that brings value? Professional ethics fundamentally seeks to answer that question and guide our projects as engineers to provide socially valuable services. In our point of view, the proposition of autonomous driving can be beneficial to increase the opportunities for mobility; as it allows people that have been previously alienated from driving because of physical limitations (i.e. people with visual impediments) to have an independent way of moving about. Likewise, autonomous navigation can open the door for more units of public transport and better traffic control.

On the other hand, when actually considering the logistics of implementing autonomous driving, the ethical argument becomes more complex. Driving is largely a social interaction, where drivers must decide how to interact with each other in the safest manner possible and decide how to react in tricky situations. Most computational algorithms, including many forms of AI, lack this ability. This is shown by the numerous ethical gaps that confine our solution to the status of prototype. For example, the controller only has knowledge of the inputs we give it: lane position and detected signs, and it will follow these conditions rigorously, if a pedestrian were to cross when there is a green light there is no feasible way for our system to detect this, and it would result in an accident.

This is an example of how human nature–i.e. not following rules, behaving unpredictably–adds a lot of complexity to engineering systems, that must nevertheless be taken into account, if we desire to create an autonomous system that can guarantee safety to some degree in the real world. Taking full control

from the driver puts the moral burden on the algorithm running inside the car's processor, or rather, on those who designed it, wrote it and validated it. They are thus held accountable for the deterministic way the car must handle any situation it is put in, no matter how ethically complex it could be, since it is because of them that the car would end up taking any particular course of action and assuming any corresponding consequences.

Because of the above, we consider that the correct ethical stance to take regarding self-driving is to recognize that it is a goal worth working towards, but the current technology is simply not enough to consider all the challenges that need to be solved. There are already several legal cases that show that the driver, who must be aware of the degree of effectiveness of the car's technology, can also be liable for any damage the car may cause, so for the foreseeable future, the ethical dilemmas faced by those making and buying these first self-driving cars will still have humans at the center.

6 Conclusion

The present work shows how the self-diving domain combines knowledge from a great variety of fields, namely computer vision, control theory, and machine learning. It also provides basic solutions to road detection and signal classification tasks. We show how to improve basic control techniques and visual servoing by applying the Kalman Filter as a linear state estimator to mitigate the noise present in the image and the sigmoid function to constrain the control output to a usable range.

7 Individual Contributions

All authors contributed equally to the project.

References

1. Townsend, A. (2020). The 100-year history of self-driving vehicles. Retrieved from https://onezero.medium.com/the-100-year-history-of-self-driving-vehicles-10b8546a3318
2. Becker, A. (2023) Kalman Filter From The Ground Up. First.
3. Jocher, G. (2020). YOLOv5 by Ultralytics (Version 7.0) [Computer software]. https://doi.org/10.5281/zenodo.3908559

A Collected Dataset

https://universe.roboflow.com/itesm-easou/navigateit_v2

B Code base

https://github.com/PAOK-2001/NavigateIt

C Video

https://www.youtube.com/watch?v=Uu2hOySvr2M&t=11s